

NEUROHACKADEMY 2026

Using GitHub Ecosystems to Your Advantage

by Serge Koudoro
Research Software Engineer
Indiana University

ROADMAP

Table of Contents

01	Six GitHub Tools, Explained	Releases · Packages · Actions · Codespaces · Pages · Copilot
02	Case Study: A Brain-Scan Pipeline	Putting the tools together, with the real repo
03	Hands-on Student Lab	Five guided steps, start to finish
04	What's Free (and What Isn't)	Limits, and how to work around them
05	Recap & Takeaways	What you learned, and where to go next

WHAT YOU'LL WALK AWAY WITH

By the end, you'll be able to...

- Explain what each of the six GitHub tools does, in plain terms.
- Publish live, always-fresh results with GitHub Pages instead of static files.
- Build a pipeline that stores data, builds its own environment, and runs itself.
- Know the free-tier limits before you hit them, and how to work around each one.

GitHub Releases

your file storage

WHAT IS IT?

A labeled snapshot of your project where you can attach big files — datasets, installers, archives — that are too heavy for a normal commit.

HOW DO YOU USE IT?

Tag a version like `v1.0`, click "Draft a new release," drag your files in, and hit publish. Each file gets its own download link.

FREE VS. PAID

Same for everyone: as many files as you want, but each one tops out at **2 GB**. No paid plan raises this — bigger files just get split up.

HOW WE'LL USE IT

In our repo, a Release tagged `v1.0-data` holds `sample_dmri.nii.gz` instead of it living in Git.

GitHub Packages (GHCR)

your app's toolbox

WHAT IS IT?

GitHub's own storage shelf for Docker containers — think of it as a box holding all the software your project needs to run, ready to reuse.

HOW DO YOU USE IT?

Build your container, tag it (like `ghcr.io/you/app:1.0`), and push it up. From then on, anything can pull that exact same environment.

FREE VS. PAID

Public projects: unlimited, free. Private projects on the free plan get **500 MB** of storage and **1 GB** of transfer a month — paid plans raise both.

HOW WE'LL USE IT

Our repo's `Dockerfile` gets built and pushed to GHCR automatically — named after the repo, no manual renaming needed.

GitHub Actions

your robot helper

WHAT IS IT?

A robot that runs jobs for you automatically — when you push code, on a schedule, or with one click. No server of your own required.

HOW DO YOU USE IT?

Add a simple text file under `.github/workflows/` that says what to run and when. GitHub does the rest.

FREE VS. PAID

Public projects: unlimited minutes. Private projects on the free plan get **2,000 minutes** a month, then pay per minute; paid plans include more.

HOW WE'LL USE IT

Two workflows do the work:

`build-container.yml` builds our image, and
`run-pipeline.yml` runs it and publishes results.

GitHub Codespaces

your coding computer

WHAT IS IT?

A full coding setup that runs in the cloud and opens right in your browser — nothing to install, and it's the same for every student.

HOW DO YOU USE IT?

List what you need in a file called `devcontainer.json`, then click "Code → Codespaces → Create codespace."

FREE VS. PAID

Free for everyone: **120 hours** plus **15 GB** of storage a month. Paid plans and school/org accounts get more, plus pay-as-you-go.

HOW WE'LL USE IT

Our repo's `.devcontainer/devcontainer.json` gives every student the identical sandbox — no setup, no "it works on my machine."

GitHub Pages

your free website

WHAT IS IT?

Free website hosting, straight from a folder in your project — no separate web server to set up or pay for.

HOW DO YOU USE IT?

Go to "Settings → Pages," pick a branch like `gh-pages` , push your HTML — GitHub builds and serves the site for you.

FREE VS. PAID

Free on every plan for public projects (private ones need a paid plan). Sites are capped around **1 GB** , with about **100 GB** of traffic a month.

HOW WE'LL USE IT

Every run publishes our QC report live at `<owner>.github.io/dmri-pipeline-test`.

GitHub Copilot

your AI pair programmer

WHAT IS IT?

An AI assistant built into your editor that suggests code, explains errors, and can chat through a problem with you as you work.

HOW DO YOU USE IT?

It's built right into Codespaces and VS Code — just start typing and accept suggestions, or open Copilot Chat to ask questions in plain English.

FREE VS. PAID

Normally a paid add-on — but it's **free for verified students and teachers** through GitHub Education. Apply at github.com/education.

HOW WE'LL USE IT

We'll lean on Copilot in our Codespace to speed up writing and debugging our processing scripts.

CASE STUDY

A Brain-Scan Quality Check, Fully Automated

Brain scan files are huge and need constant checking to make sure they're usable. Let's build a system that does that automatically, using only the six tools we just covered.

THE REAL REPO

Every File Has a Job

This is a public **template** repo — github.com/Neurohackademy2026/dmri-pipeline-test . You won't edit it directly; you'll click "Use this template" to get your own copy in Step 1.

Dockerfile

Defines the software environment. **Packages** builds and stores it as a container.

.github/workflows/build-container.yml

An **Actions** workflow that builds the Dockerfile and pushes it to GHCR.

.github/workflows/run-pipeline.yml

The other **Actions** workflow: runs the pipeline, builds the report, and deploys it to Pages.

process_dmri.py

The actual science: loads a scan, computes SNR and FA, writes the QC images and stats.

.devcontainer/devcontainer.json

Describes your dev setup. **Codespaces** reads this to build your coding sandbox.

A Release (v1.0-data)

Not a file in the repo, but a **Releases** entry you create, holding `sample_dmri.nii.gz`.

BEFORE & AFTER

A Better Way to Work

THE OLD WAY

Scattered and fragile

Data lives on different computers everywhere, making it hard to share or collaborate.

Every setup is different, so scripts that work on one machine break on another.

THE GITHUB WAY

One place for everything

Data, software, automation, dev tools, and the website all live under one project.

Everything works the same way every time, because it's defined in simple config files.

Releases Hold Our Brain Scans

Git can't handle multi-gigabyte scan files well, and Git LFS gets expensive fast with strict caps. Releases give us a better home for them.

One Zip File Per Release

Pack the raw scan files and their measurement data together into one archive.

Locked, Versioned Inputs

Each release is tagged (like `v1.0-dataset`), so we always know exactly which data we ran.

Fast Downloads

GitHub's own servers deliver both our code and our data, quickly and reliably.

One Container, Same Every Time

Build It Once

We bundle our brain-imaging software into one lightweight container, then push it to GHCR.

Tag It Like a Version

We point to a specific tag (e.g. `ghcr.io/org/dmri:1.2.0`) so nothing changes underneath us by surprise.

Starts in Seconds

Reusing the cached container cuts our startup time from 20 minutes down to under 60 seconds.

100%

STEP 3 · RUNNING THE JOB

Automated, Start to Finish

Nothing to babysit, no servers to maintain

GitHub Actions ties everything together. On a schedule, it checks for new data and kicks off the whole job automatically.

No Server Needed

Everything runs on GitHub's own cloud machines.

Grabs Our Container

Pulls our custom container from GHCR to process the scan data.

Saves the Results

Writes stats, masks, and images out to a results folder.

Summary

Who Does What in Our Pipeline

Tool	Job in Our Project	Why It Matters
Releases	Stores our raw scan files	Keeps huge files out of Git entirely
Packages	Holds our container image	No slow reinstalling of software
Actions	Runs the whole pipeline	Fully automatic, no manual steps
Codespaces	Our coding environment	Instant setup, right in the browser
Pages	Hosts our results dashboard	Shares charts with anyone, instantly
Copilot	Helps us write & debug code	Free for students & teachers

STEP 4 · SHARING THE RESULTS

A Live Dashboard, Not a PDF

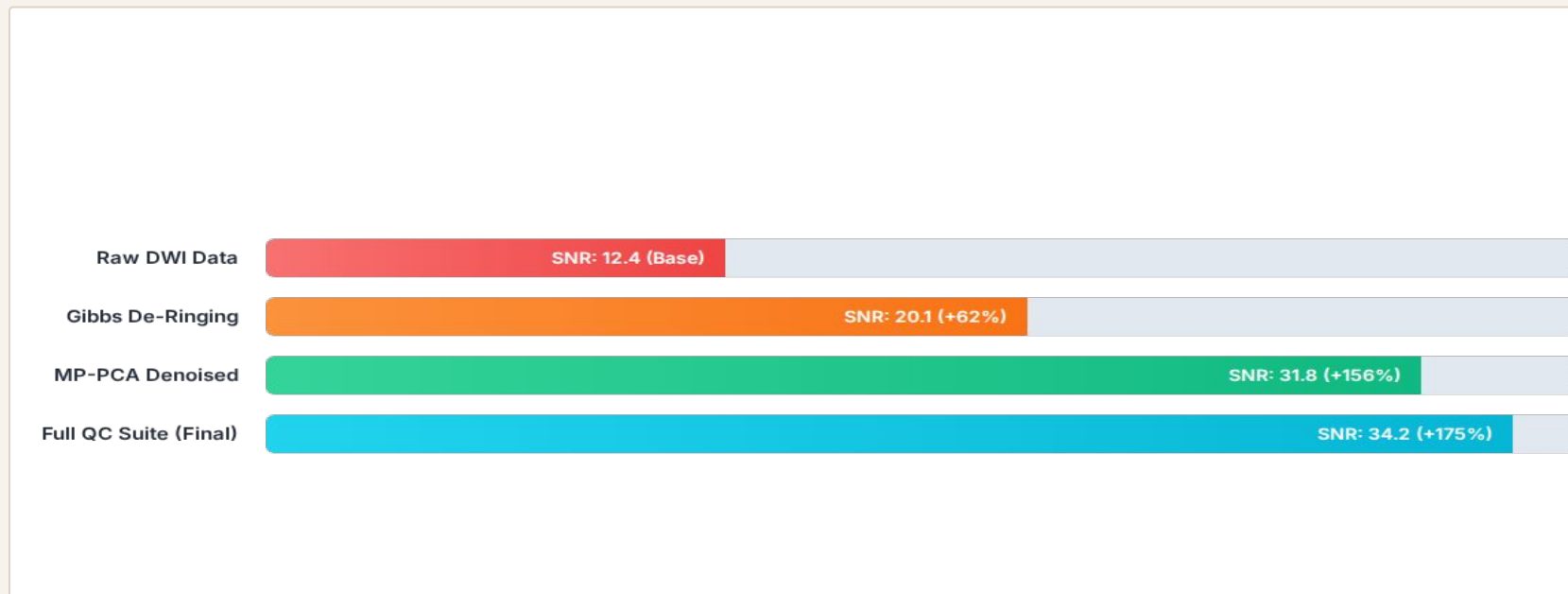
Always up to date

No more emailing static PDF files around. After each run, Actions turns the results into a simple web page and Pages hosts it instantly — charts and visuals, viewable from any browser, anywhere.

RESULTS

Better Signal, Automatically

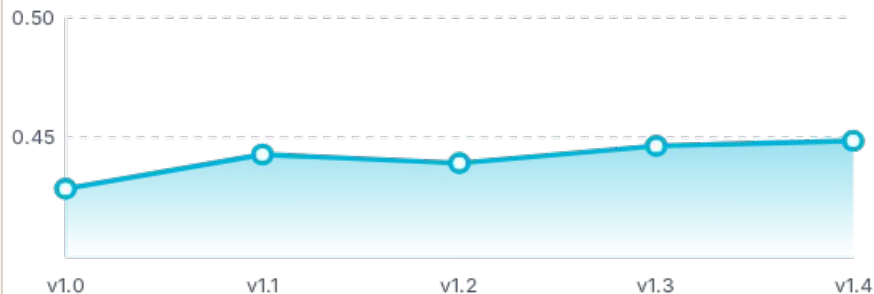
Every run compares signal quality across different processing settings, so we can see which choices actually improve the data.



TRACKING QUALITY OVER TIME

Watching the Numbers Stay Stable

Tracking Mean Fractional Anisotropy (FA) across successive automated processing rounds validates model stability.



We track a key brain-tissue measurement across every automated run, to confirm our results stay consistent.

WHY THIS MATTERS

If this number drifts unexpectedly, it's an early warning that something in the data or pipeline needs attention.

HOW IT ALL FITS TOGETHER

The Full Cycle, in Four Steps

1

Trigger

A weekly timer goes off, or new data is uploaded.

2

Set Up

A fresh machine starts, grabs our container and the new data.

3

Crunch

It processes the scans and calculates our quality metrics.

4

Publish

The dashboard updates itself and goes live on Pages.

CODING WITHOUT THE HASSLE

Skip the Local Setup Headache

Getting all the right software installed locally is a pain, especially when everyone's computer is different. Codespaces skips all of that.

Describe It Once

List what you need in a `devcontainer.json` file in your project.

Open It Anywhere

Get a full coding workspace in your browser, preloaded with everything you need.

Powerful When Needed

Run heavier jobs on strong cloud hardware, with just one click.

STAYING SAFE

GitHub Watches Your Back

KEEPING SOFTWARE UP TO DATE

Dependabot

Automatically checks our software for known issues. If something needs a fix, it opens a pull request, tests it, and gets it ready to merge.

CATCHING PROBLEMS EARLY

Secret & Code Scanning

GitHub scans for exposed passwords or risky code before it ever gets merged — important when handling sensitive data.

STAYING CONNECTED

GitHub Discussions

Ask & Answer

A place for questions, so bug reports stay focused on actual bugs.

Suggest Ideas

Let people vote on the features they'd actually want next.

Talk It Through

Share proposals and get feedback before making big changes.

YOUR TURN

Hands-on Student Lab

Let's build the whole thing together, using the real template repo. You'll copy the project, set it up, run it, and even make your own tweak to it.

Template: github.com/Neurohackademy2026/dmri-pipeline-test

1

STEP 1

Get Your Own Copy

Use the Template

Open github.com/Neurohackademy2026/dmri-pipeline-test and click the green "Use this template" button to make your own copy.

Make It Public

Set the repository to "Public." Public projects get unlimited free run time and no storage limits.

Open a Codespace

Click "Code → Codespaces → Create codespace on main." You'll have a full coding sandbox in about 20 seconds.

2

STEP 2

Build the Container

Before we can process any data, we need to build and save our container.

It Usually Builds Itself

`build-container.yml` runs automatically on a push to main that touches the Dockerfile.

Or Trigger It Yourself

Open "Actions," pick the build workflow, and click "Run workflow" to build it on demand.

Nothing to Rename

The image name comes automatically from your repository — no package names or paths to edit.

3

STEP 3

Upload the Data

Create a Release

On your repository's main page, find "Releases" and click "Create a new release."

Tag It Correctly

Name the tag exactly `v1.0-data`. This tells the pipeline where to find our data.

Attach the File

Drag in `sample_dmri.nii.gz` and click "Publish."



A Good Rule to Remember

Never commit big data files directly into git — it slows down the whole project for everyone.

4

STEP 4

Run It & Deploy to Pages

1. Allow Writes

Settings → Actions → General.
Grant "Read and write permissions."

2. Run It

Actions → "Execute dMRI Pipeline" → "Run workflow."
Wait for success.

3. Turn On Pages

Settings → Pages → Build and deployment → Source → "GitHub Actions."

4. Go Live

Your report is live at
`<owner>.github.io/dmri-pipeline-test` .



OPTIONAL EXTRA

Optional: Email Notifications

Our `run-pipeline.yml` workflow includes a **notify** job that can email you the results — signal quality, status, and a link to the report — after every run. It's completely opt-in: if you skip the setup below, it's automatically skipped too, and nothing breaks.

SET UP IN 4 SECRETS

Goto Settings → Secrets and variables → Actions → New repository secret and add:

MAIL_USERNAME the sender's email address

MAIL_PASSWORD an app password, not your real one (for Gmail: Google Account → Security → 2-Step Verification → App Passwords)

MAIL_TO who should receive the emails

MAIL_SERVER / PORT optional — defaults to `smtp.gmail.com / 465`

UNDER THE HOOD

The notify job uses the `dawidd6/action-send-mail` GitHub Action to send the message — a ready-made tool, no email code to write yourself. No secrets set? The job quietly skips itself. Your pipeline still runs and publishes to Pages exactly as before.

5

STEP 5

Make It Your Own

EDIT LOCALLY (IN YOUR CODESPACE)

Tweak the Code

Open `process_dmri.py` in your Codespace.

Try changing a plot setting (like `cmap='hot'`) or a threshold (like `snr > 10`).

SHIP IT INSTANTLY

Commit & Push

Commit your change and push it to main — just like any other update.

The pipeline fires automatically, recalculates everything, and updates the live dashboard in about 30 seconds.

WHAT'S REALLY FREE?

Free, With a Few Catches

Yes, this whole pipeline can run for free — but there are real limits worth knowing before you build around them.

THE NUMBERS

Free Tier Limits, at a Glance

Tool	Free Limit	Where It Bites Us
Actions	Unlimited (public), 2,000 min/mo (private)	No GPUs; capped at 2 CPUs & 7 GB RAM
Releases	Unlimited files, 2 GB max each	Big datasets must be split into chunks
Packages	Unlimited (public), 500 MB (private)	Our container with FSL/MRTrix3 can top 3 GB
Pages	Free on public repos, 1 GB site cap	Not available on free private repos
Codespaces	120 core-hours / month	Heavy 3D visuals can burn through it fast

WORKING AROUND THE LIMITS

Simple Fixes That Cost Nothing

Bring Your Own Computer

Connect a lab computer or university cluster as a "self-hosted runner" to skip the standard limits entirely, for free.

Split Big Files

If a file is over 2 GB, break it into smaller pieces and have the pipeline stitch them back together.

Keep It Public

Public repositories skip Actions time, Releases bandwidth, and Packages storage fees entirely.

Trim Your Container

Leave out anything you don't actually need at runtime, to keep images small and fast to load.

RECAP

What Did You Learn?

Releases

Store big files outside of Git.

Packages

Package your tools, once and for all.

Actions

Automate everything, no server needed.

Codespaces

Code instantly, from any browser.

Pages

Publish results the moment they're ready.

Copilot

Free AI help for students & teachers.

The Big Idea

Six ordinary GitHub tools, combined, can replace an entire research computing setup — storage, computing, coding environment, dashboard, and security — for free, on public projects.

GO BUILD SOMETHING

Thanks for Learning With Me

You now know six tools that turn GitHub into a full research computing platform — free, reliable, and reproducible. Go try it on your own project.

Get your own copy — it's a template:

github.com/Neurohackademy2026/dmri-pipeline-test